First Two-Class Appointment Simulation: Recommended Formulation

Alexandre SEPULVEDA de DIETRICH
Under the supervision of Pr. Carri Chan

April 8, 2026

1 Purpose

This note defines the first discrete-event appointment simulation used in this project and clarifies the notation so it matches the underlying dynamics of the problem. The main design goal is to keep the first simulator simple enough to run, inspect, and extend, while using a formulation that remains valid once we add richer allocation policies, heterogeneous patient behavior, or optimization layers.

The note aligns the simulator with the queueing models in:

- Green and Savin (2008), *Reducing Delays for Medical Appointments: A Queueing Approach*,
- Liu and Ziya (2014), *Panel Size and Overbooking Decisions for Appointment-based Services under Patient No-shows*,
- Liu (2015), *Optimal Choice for Appointment Scheduling Window under Patient No-Show Behavior*.

All three papers study the same structural mechanism:

more backlog $\Rightarrow$ more delay $\Rightarrow$ worse attendance $\Rightarrow$ lower effective capacity.

The simulation keeps that mechanism, but implements it on an explicit rolling calendar with two patient classes, explicit cancellations, and a pluggable slot-allocation rule.

2 Why the Naive Formulation is Not Sufficient

An initial formulation might write

$$X_{i,\tau}^{D} = \text{number of class-}i\text{ patients booked for day } D+\tau$$

and then also use τ inside $b_i(\tau)$ and $\xi_i(\tau)$. That notation is not the best formulation for this problem.

2.1 Problem 1: It Overloads the Delay Variable

There are two different delays in the system:

1. the *remaining delay* from the current day to the appointment,
2. the *original delay seen at booking time*.

These play different roles. The remaining delay is a state variable. The original delay at booking is the natural argument of the behavioral functions.

If the same symbol τ is used for both, then the model becomes ambiguous. In particular, it can incorrectly suggest that no-show probability should be recomputed from remaining delay as the appointment approaches. That is not the intended behavioral mechanism here.

2.2 Problem 2: The Aggregate Day-Level Count is Not the True State

The vector $\{X_{i,\tau}^{D}\}$ is a useful summary, but it is not the full simulator state once we include:

- slot-level capacity,
- day-by-day cancellations that reopen specific future slots,
- alternative booking policies that may choose different open slots within the same day.

For those features, the actual state must retain the slot-level calendar.

2.3 Problem 3: It Hides the Link to the Literature

In the source papers, attendance depends on the backlog or delay the patient sees when the appointment is accepted. The relevant quantity is therefore the *booking-time delay*, not the residual delay just before service.

The simulator should preserve that same logic in explicit-calendar form.

3 Recommended Formulation

3.1 Decision Epochs

Let time advance in slot-sized decision epochs. Write

$$t = (D, s),$$

where:

- D is the calendar day,
- $s \in \{0, 1, \ldots, S-1\}$ is the active slot within day D.

In the first version of the simulator,

$$H = 15 \text{ days}, \qquad S = 25 \text{ slots per day},$$

where H is the booking horizon.

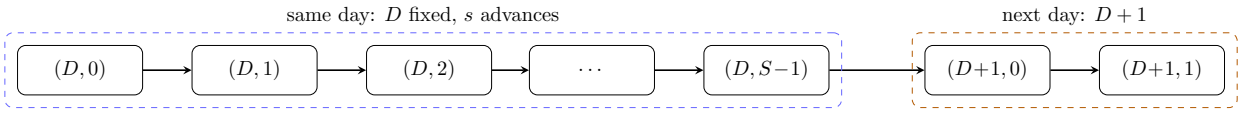
3.2 How Time Advances

The ordered pair $t = (D, s)$ advances lexicographically:

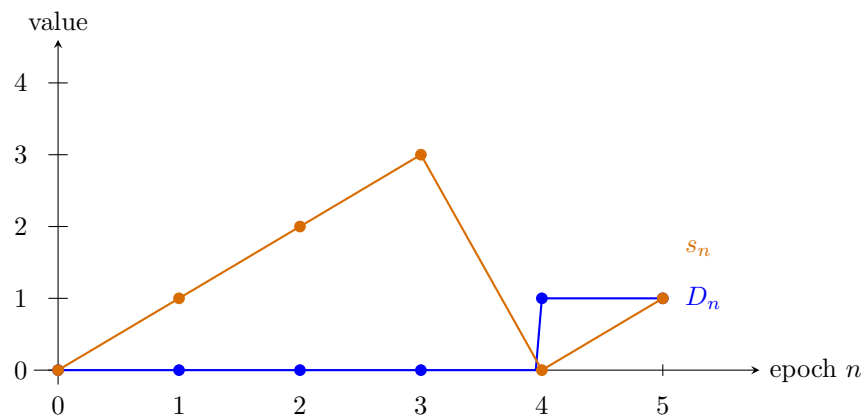
$$(D, s)^{+} = \begin{cases} (D, s+1), & \text{if } s < S-1, \\ (D+1, 0), & \text{if } s = S-1. \end{cases}$$

So the slot index s moves within a day, and the day index D increases only when the current day has exhausted all S slots.

The same evolution can be visualized as a state-transition graph:



To see both coordinates on the same graph, it is useful to plot them against the epoch index. The following illustration starts at day 0 and uses $S = 4$ only so the rollover is visible on a small plot:



On this graph, D_n is a step process while s_n increases within a day and resets to 0 when the process rolls to the next day.

This is the time scale on which arrivals, booking decisions, cancellations applied at the start of each day, and service realizations are processed.

3.3 True State: Slot-Level Calendar

The true simulator state at time $t = (D, s)$ is the rolling slot-level calendar. For each residual day offset

$$r \in \{0, 1, \ldots, H-1\}$$

and each slot index

$$m \in \{0, 1, \ldots, S-1\},$$

define

$$Y_t(r, m).$$

Interpretation:

- $Y_t(r, m) = 0$ if slot $(D + r, m)$ is currently open,
- $Y_t(r, m) = (i, \tau, c)$ if slot $(D + r, m)$ is occupied by a class-i patient whose original offered delay was τ, with cancellation marker c.

The cancellation marker c is either:

- a future calendar day on which the appointment will be canceled before service, or
- $\varnothing$ if the appointment will not pre-cancel.

This is the minimal state representation that supports:

- explicit holes created by cancellation,
- slot-level FCFS or non-FCFS allocation,
- class-specific booked-delay tracking,
- later policy experiments that depend on slot position.

3.4 Derived Day-Start Summary

Although the slot-level calendar is the true state, it is still useful to retain an aggregate day-start summary that resembles the literature. At the start of day D, after applying cancellations for day D and before processing slot 0, define

$$X_{i,r}^{D} = \sum_{m=0}^{S-1} \mathbf{1}\{Y_{(D,0)}(r,m) \text{ is occupied by a class-}i \text{ patient}\},$$

for

$$i \in \{1,2\}, \qquad r \in \{0,1,\ldots,H-1\}.$$

Interpretation:

$$X_{i,r}^{D} = \text{number of class-}i \text{ patients scheduled at the start of day } D \text{ for day } D+r.$$

This is the correct interpretation of the state-count variable. The index r is a *residual delay*, not a booking-time delay.

3.5 Worked Example of Y and X

It is useful to see one concrete calendar snapshot. Suppose:

$$D = 20, \qquad s = 0, \qquad H = 3, \qquad S = 4.$$

So we are at the start of day 20, before processing the first slot of that day, and the rolling calendar covers days 20, 21, and 22.

Assume the slot-level state is:

	$m=0$	$m=1$	$m=2$	$m=3$
$r=0$	0	$(1,0,\varnothing)$	$(2,0,\varnothing)$	0
$r=1$	$(1,1,\varnothing)$	0	$(1,1,\varnothing)$	$(2,1,\varnothing)$
$r=2$	0	$(2,2,21)$	$(1,2,\varnothing)$	0

Interpretation:

- row $r=0$ corresponds to today's remaining slots, that is, day 20,
- row $r=1$ corresponds to day 21,
- row $r=2$ corresponds to day 22,
- entry 0 means the slot is open,
- entry (i,τ,c) means the slot is occupied by a class-i patient who accepted an offered delay τ, with cancellation marker c.

For example:

- $Y_{(20,0)}(1,0) = (1,1,\varnothing)$ means slot $m=0$ on day 21 is booked by a class-1 patient who, when booking, accepted a one-day delay,
- $Y_{(20,0)}(2,1) = (2,2,21)$ means slot $m=1$ on day 22 is booked by a class-2 patient who accepted a two-day delay and is scheduled to cancel on day 21 before service.

From this calendar, the derived day-start counts are:

$$X_{1,0}^{20} = 1, \quad X_{2,0}^{20} = 1,$$

$$X_{1,1}^{20} = 2, \quad X_{2,1}^{20} = 1,$$

$$X_{1,2}^{20} = 1, \quad X_{2,2}^{20} = 1.$$

So the vector $X_{i,r}^{20}$ is just a class-by-class count of the occupied entries in each row. It does *not* remember which slot inside the day is occupied, nor does it retain the cancellation marker. That is why $X_{i,r}^{D}$ is a useful summary, but not the full state needed by the simulator.

The distinction between r and τ is also visible in this example. Consider the patient stored as

$$Y_{(20,0)}(2,2) = (1, 2, \varnothing).$$

On day 20, that patient contributes to $X_{1,2}^{20}$. If the patient remains scheduled, then at the start of day 21 the same patient will contribute to $X_{1,1}^{21}$, because the *residual delay* has fallen from 2 to 1. However, the patient's booking-time delay remains $\tau = 2$, and that is the quantity that should continue to drive the behavioral functions $b_1(\tau)$ and $\xi_1(\tau)$.

3.6 Booking-Time Delay

Now define the behavioral delay variable separately:

$\tau =$ delay in days between the booking day and the appointment day offered to the patient.

If a patient arrives on day d and is offered a slot on day $d + \tau$, then:
- $b_i(\tau)$ is the probability that class-i declines the offer,
- if the patient accepts, τ is stored with the booking,
- $\xi_i(\tau)$ is the no-show probability used at appointment time if the patient remains scheduled.

This separation is the key modeling choice. Residual delay belongs to the state. Booking delay belongs to the behavioral rules.

4 Patient Classes and Arrival Mix

The class-specific behavior of each class $i \in \{1, 2\}$ is described by

$$B_i = \{b_i(\tau), \phi_i, \xi_i(\tau)\}.$$

In the current two-class prototype, arrivals are parameterized at the system level by

$$\lambda = \text{total expected arrival rate per slot}, \qquad p \in [0, 1] = \text{share of class-1 demand}.$$

The class-specific arrival rates are then induced by

$$\lambda_1 = p\lambda, \qquad \lambda_2 = (1 - p)\lambda.$$

Equivalently, the full class objects can still be written as

$$C_i = \{\lambda_i, b_i(\tau), \phi_i, \xi_i(\tau)\},$$

but in the first simulation experiments the primitive demand parameters are (λ, p) rather than (λ_1, λ_2).

The interpretation is therefore:
- λ: total expected arrival rate per slot,
- p: fraction of arrivals belonging to class 1,
- λ_i: implied arrival rate of class-i requests per slot,

- $b_i(\tau)$: probability the patient declines the offered slot because the offered delay is too large,
- ϕ_i: one-shot pre-appointment cancellation probability after booking,
- $\xi_i(\tau)$: no-show probability at appointment time, conditional on the patient still being scheduled.

In the current first version, ϕ_i is constant within class. Later versions may generalize this to $\phi_i(\tau)$ if cancellation is also delay-sensitive.

5 Event Dynamics

5.1 Slot-Level Event Order

At each active slot (D, s), the simulator executes the following steps.

1. Apply all cancellations whose cancellation day is D.
2. Generate arrivals
$$A_i(D, s) \sim \text{Poisson}(\lambda_i), \qquad i \in \{1, 2\}.$$
In the current parameterization, $\lambda_1 = p\lambda$ and $\lambda_2 = (1 - p)\lambda$.
3. Randomize the order of individual arrivals within the slot.
4. For each arrival, call the allocation policy to select an open future slot.
5. If a slot on day $D + r$ is offered, interpret
$$\tau = r.$$
6. The patient accepts with probability
$$1 - b_i(\tau).$$
7. If the patient books and $\tau \geq 1$, then with probability ϕ_i a cancellation day is sampled uniformly from the days strictly before the appointment day.
8. When the active slot is served, the patient in that slot, if any, attends with probability
$$1 - \xi_i(\tau),$$
where τ is the original delay stored when the patient booked.

5.2 FCFS Baseline Policy

In the baseline version, the allocation rule is:

> choose the earliest open slot strictly after the active slot.

That rule is first-come first-served in calendar order, but the simulator is intentionally written so the slot-selection rule can later be replaced without changing the event loop. The current codebase now includes two additional initial-model examples:

- a strict reserved-capacity policy that protects a specified fraction or count of daily slots for each class,
- a class-specific booking-window policy that restricts class i to delays no larger than k_i.

6 Performance Measures

6.1 Arrival-Cohort Measures

For patients who arrive during the measured window, define:

$$\begin{aligned}
\text{Arrivals}_i &= \text{number of class-}i\text{ appointment requests,}\\
\text{Booked}_i &= \text{number of class-}i\text{ requests that accept an offer,}\\
\text{Balked}_i &= \text{number of class-}i\text{ requests that do not book,}\\
\text{Canceled}_i &= \text{number of class-}i\text{ booked patients who pre-cancel,}\\
\text{NoShow}_i &= \text{number of class-}i\text{ booked patients who remain scheduled but do not attend,}\\
\text{Served}_i &= \text{number of class-}i\text{ booked patients who attend,}\\
\bar{\tau}_i^{\text{book}} &= \text{mean original booking delay among class-}i\text{ booked patients.}
\end{aligned}$$

The main service metric in the first version is

$$\text{ServiceRate}_i = \frac{\text{Served}_i}{\text{Booked}_i}.$$

Aggregate values are formed by summing over classes and taking the corresponding weighted mean.

6.2 Slot-Based Measures

Over measured service slots, define:

$$\begin{aligned}
\text{BookedSlots} &= \text{slots with a booked patient at service time,}\\
\text{ServedSlots} &= \text{slots where the booked patient attends,}\\
\text{NoShowSlots} &= \text{slots where a booked patient is present on the schedule but does not attend,}\\
\text{EmptySlots} &= \text{slots with nobody scheduled at service time.}
\end{aligned}$$

Two utilization measures are reported:

$$\rho^{\text{booked}} = \frac{\text{BookedSlots}}{HS_{\text{measured}}}, \qquad \rho^{\text{attended}} = \frac{\text{ServedSlots}}{HS_{\text{measured}}}.$$

The first measures scheduled load. The second measures realized attended load.

7 Comparison to the Literature

7.1 Common Structural Mapping

The source papers use more stylized queue states than the simulator, but the economic and behavioral interpretation is the same. In each paper, attendance depends on the congestion level a patient sees when entering the system.

In the analytical papers, that congestion variable is usually:

- queue length,
- backlog level,
- or the number of appointments already ahead in the system.

In the simulator, the explicit-calendar analog is the offered delay τ. That is why τ, rather than residual delay r, should drive $b_i(\cdot)$ and $\xi_i(\cdot)$.

7.2 Green and Savin (2008)

Green and Savin study a single-server appointment queue in which no-show probability increases with the backlog seen when the appointment is made. Their no-show curve is of the exponential-saturation form

$$\gamma(k) = \gamma_{\max} - (\gamma_{\max} - \gamma_0)e^{-k/C},$$

where k is a backlog measure.

The mapping to the simulator is:

- their backlog state corresponds to our explicit slot calendar,
- their delay-sensitive no-show mechanism corresponds to our $\xi_i(\tau)$,
- their stylized queue-length state is refined into the explicit schedule $Y_t(r, m)$.

The simulation differs from Green and Savin in several ways:

- it allows two patient classes rather than one homogeneous stream,
- it includes explicit balking before booking,
- it includes explicit pre-appointment cancellation timing,
- it does not yet model no-show rescheduling.

The most important conceptual similarity is this: attendance deteriorates with the delay the patient faced when entering the schedule.

7.3 Liu and Ziya (2014)

Liu and Ziya study panel size and overbooking decisions in stylized queueing models with delay-dependent attendance. Their state variable is again a queue-length-type object, and a patient's show-up probability depends on the congestion level observed at booking time.

Relative to Liu and Ziya, the simulator:

- fixes daily slot capacity at 25 in the first version,
- does not yet optimize overbooking or panel size,
- keeps the booking calendar explicit rather than compressing it into a single queue-length state,
- is designed to allow class-specific balking, cancellation, and attendance curves.

Their model is therefore more compact and optimization-oriented. Our simulator is more descriptive and state-rich. The key shared mechanism remains the same:

$$\text{booking-time congestion} \longrightarrow \text{attendance}.$$

7.4 Liu (2015)

Liu studies the optimal appointment-window size K under delay-dependent no-shows. That paper's main control is the admissible backlog or booking window.

The simulator's rolling horizon length H plays the same operational role as a hard scheduling window, but here the horizon is fixed in the baseline implementation. This makes the simulator a natural platform for later experiments in which:

- H is varied,
- different allocation policies are compared,
- or class-specific admission rules are introduced.

Again, the important mapping is not the exact state variable but the interpretation: the patient's future attendance depends on the delay accepted at booking time.

7.5 Side-by-Side Comparison

Model / paper	State abstraction	Delay variable that drives attendance	Main difference from this simulator
Green and Savin (2008)	queue length / backlog in an appointment queue	backlog seen when appointment is made	analytical single-class queue; no explicit slot calendar
Liu and Ziya (2014)	queue length in $M/M/1$ or $M/D/1$ style models	congestion at booking time	optimization over panel size and overbooking, not explicit calendar simulation
Liu (2015)	queue/window state with finite scheduling window K	delay implied by admissible backlog at booking	optimization over window size rather than fixed-horizon simulation
This simulator	explicit rolling calendar $Y_t(r, m)$ plus derived $X^D_{i,r}$	original offered delay τ	richer operational detail, two classes, explicit cancellation holes, policy plug points

8 Why this Formulation is the Most Appropriate for This Problem

This formulation is the most appropriate starting point for the present project for four reasons.

8.1 1. It Matches the Behavioral Mechanism

The patient decides whether to book when seeing an offered delay. If they book, their future no-show behavior should be tied to that accepted delay. That is exactly what τ captures.

8.2 2. It Preserves the Literature's Logic While Using an Explicit Calendar

The source papers compress the calendar into queue length for tractability. The simulator does not need to do that. It can retain the slot-level calendar and still summarize the system in a literature-friendly way through $X^D_{i,r}$.

8.3 3. It Is Markov for the Planned Extensions

Once cancellations create holes and policies can choose among open slots, a pure day-level count is no longer enough. The calendar state $Y_t(r, m)$ is the right object to carry forward if the project later adds:

- priority rules,
- reserved capacity by class,
- different appointment windows by class,
- overbooking,
- dynamic acceptance or rejection decisions.

8.4 4. It Cleanly Separates State and Behavior

The formulation distinguishes:

- *where the system is now*: represented by residual delay r in the state,
- *what delay the patient accepted*: represented by τ in the behavior functions.

That separation is the cleanest way to avoid conceptual mistakes in both code and analysis.

9 Implementation Mapping

The current codebase follows this notation directly:

- the slot calendar stores each patient's class, appointment location, booking day, and the mathematical delay τ,
- the arrival log records the offered delay as `tau_offered`,
- the cohort and slot logs record the same mathematical quantity τ under the implementation field name `tau_booked`,
- the day-start state log records the residual day offset as `residual_delay`.

This is exactly the split recommended above. In addition, the simulator now exposes a daily journal with access, booking, cancellation, attendance, and utilization measures, and reports the access metric

$$\mathbb{P}(\tau < T_{\text{access}})$$

for a configurable target such as $T_{\text{access}} = 30$ days.

10 References

- Green, L. V., and S. Savin. 2008. Reducing delays for medical appointments: A queueing approach. *Operations Research* 56(6):1526–1538.
- Liu, N. 2015. Optimal choice for appointment scheduling window under patient no-show behavior. *Production and Operations Management* 25(1):128–142.
- Liu, N., and S. Ziya. 2014. Panel size and overbooking decisions for appointment-based services under patient no-shows. *Production and Operations Management* 23(12):2209–2223.